

机器学习

第一讲

薛薇

办公地点：明主1013

E-Mail: xuewei@ruc.edu.cn

主要内容

- 讨论以几个问题
 - 问题1：课程内容的定位和考核？
 - 机器学习与人工智能
 - 问题2：数智时代统计学(与数据打交道)专业开设机器学习课程的原因？
 - 统计学与人工智能中机器学习的内在联系？目标？
 - 问题3：经典统计方法与机器学习算法的互补性？
 - 经典统计方法的优势和不足？学习机器学习算法的必要性？

问题1：课程内容的定位和考核

- 人工智能 (Artificial Intelligence, AI)，诞生于20世纪50年代，旨在实现人脑部分思维的计算机模拟，完成人类智力任务的自动化实现
 - **目标：**实现通用人工智能---会听（语音识别和机器翻译等）、会看（文字识别和图像识别等）、会说（语音合成和人机对话等）、会学习（知识表示和机器学习等）、会思考（逻辑推理和人机对弈）和会行动（机器人和自动驾驶等）
 - **机器学习** (Machine Learning) 是人工智能发展的一个阶段
- 课程内容定位：**非人工智能通识课**



- 引入DeepSeek/文心一言等，强调“去算法化”、“零代码”、“重AIGC(人工智能生成内容 Artificial Intelligence Generated Content) ”
- **课程目标：**培养解决**数据分析**实际问题的能力，需要：
 - 可解释性需：“**重基本原理**”
 - 分析手段需：“**适度代码**”

课上重点：

- 方法基本原理的讲解(有些章节不讲)
- 授课方式：理论+适度编程+应用

- 建议安装Python集成开发环境：

Anaconda

- 建议使用Jupyter Notebook

- .ipynb文件

课下重点：

- 阅读Python代码温习和加深对基本原理的理解
- 基于Python进行案例数据分析



```
In [1]: 1 import pandas as pd
        2 df=pd.read_excel('北京市空气质量数据.xlsx')
        3 data=df.head()
        4 print('获取到所有的值:\n{0}'.format(data))
```

获取到所有的值：

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3
0	2014-01-01	81	良	45	111	28	1.5	62	52
1	2014-01-02	145	轻度污染	111	168	69	3.4	93	14
2	2014-01-03	74	良	47	98	29	1.3	52	56
3	2014-01-04	149	轻度污染	114	147	40	2.8	75	14
4	2014-01-05	119	轻度污染	91	117	36	2.3	67	44

说明：1、利用Pandas的read_excel将一个关于空气质量监测数据的Excel数据文件读到数据框 (DataFrame)中。2、head默认读取前五行观测数据。3、print输出所读取的前五行观测数据。

问题1：课程内容的定位和考核

- 教材和参考书

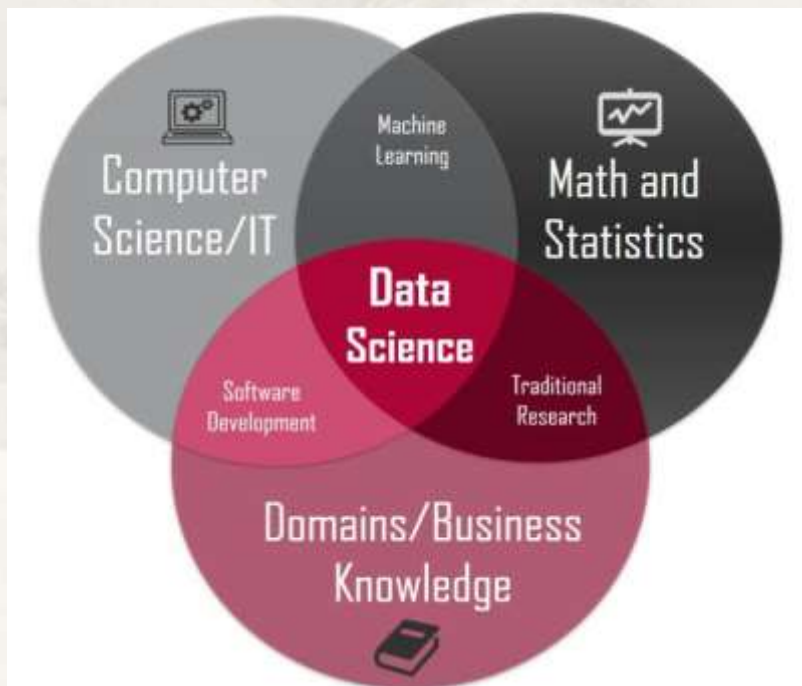
- 薛薇,《Python机器学习：原理与实践(第2版)》，中国人民大学出版社，2024.1
- Jack VanderPls,《Python Data Science Handbook》，2017
- 周志华,《机器学习》，清华大学出版社,2015

- 考核方式：

- 平时成绩(平时作业(70%)+小测(30%))：**40%**
 - 考察：原理掌握程度、分析问题的逻辑性、完整性和反思能力、适度代码能力
- 期末考核方式：个人大作业：**60%**
 - 考察：独立发现问题、解决问题、反思和方案迭代能力、适度代码能力、基本文字素养

问题2：统计学专业开设机器学习课程的原因？

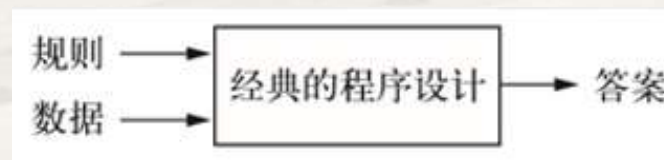
- 2019年1月20日华为创始人兼CEO任正非在接受央视《面对面》采访,当谈到人工智能时他表示,人工智能就是统计学,计算机与统计学就是人工智能(机器学习), **如何理解?**
 - **理解1：学者眼中的机器学习：看机器学习与统计学科的内在联系**
 - 2010年美国机器学习专家德鲁·康威(Drew Conway) 提出的维恩图 (Venn Diagram, 又称范氏图)



- 例：MIS系统的软件开发，等
- 例：问卷调查；以概率论和推断统计为基础的传统统计分析，等
- 机器学习具有浓厚的**跨学科**色彩
 - 需要：**“重基本原理”**
 - 需要：**“适度代码”**
 - 需要：**业务场景**
- 机器学习成为数智时代统计学科的重要知识构件
 - **专业核心课**

问题2：统计学专业开设机器学习课程的原因？

- 2019年1月20日华为创始人兼CEO任正非在接受央视《面对面》采访,当谈到人工智能时他表示,人工智能就是统计学,计算机与统计学就是人工智能(机器学习), **如何理解?**
 - **理解2：**从人工智能的发展：看机器学习与统计学科目标的一致性
 - 人工智能的发展：**符号主义人工智能** (symbolic AI)、机器学习、深度学习，等
 - 符号主义人工智能(弱人工智能)：基于“一切都可规则化编码”的基本信念：通过硬编码，依指定规则自动完成相应的处理任务(例：依据BMI(Body Mass Index)判断肥胖)
 - 棋类对弈程序、专家系统
 - 符号主义人工智能很难解决没有明确规则定义的问题
 - 例：不能仅依据BMI对肥胖进行判断



问题2：统计学专业开设机器学习课程的原因？

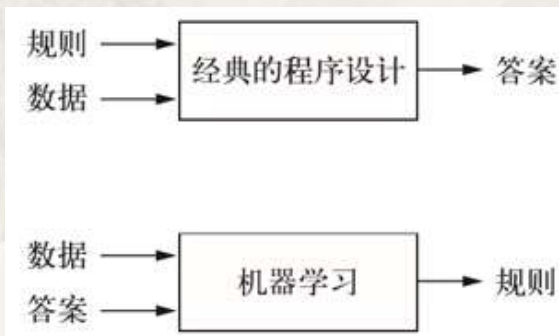
■ 理解2：从人工智能的发展：看机器学习与统计学科目标的一致性

- 人工智能的发展：符号主义人工智能 (symbolic AI)、机器学习、深度学习，等
- 机器学习的提出：人工智能之父阿兰·图灵的图灵测试：“思考的机器”是可能的
- 机器学习：
 - 目的：教计算机借助某些算法完成任务
 - 一种新的编程范式(相对于经典的程序设计)
 - 例：对是否肥胖的判断



- 1912.6.23~1954.6.7终年42岁
- 研发图灵机(与冯·诺伊曼机齐名)
- 二战期间参与破译德军密码(Enigma 埃尼格码)的ULTRA(厄尔特拉)计划
- 2009年，英国首相戈登·布朗代表英国政府正式向图灵致歉：图灵受到的对待是“骇人听闻的”和“完全不公平的”

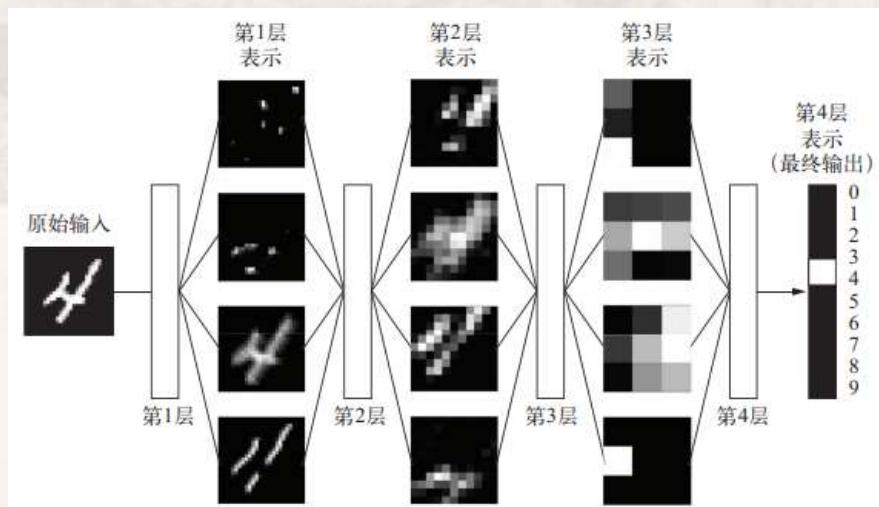
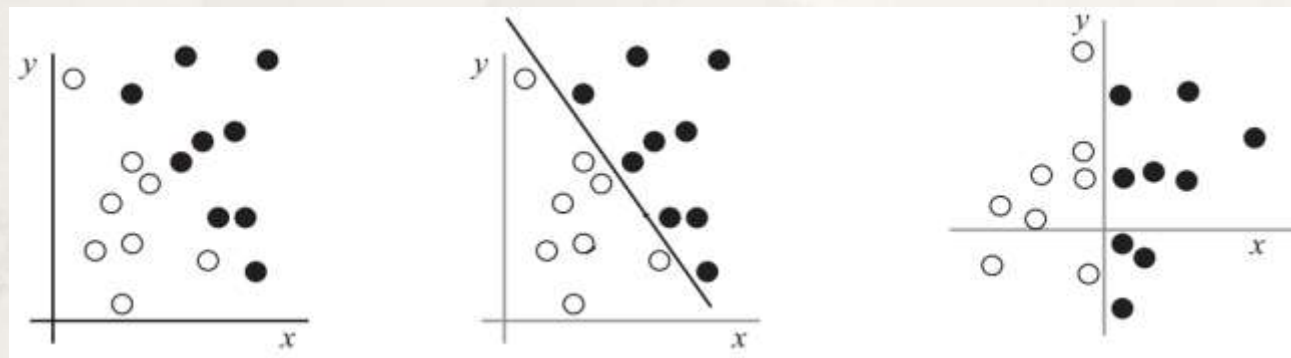
体脂率、性别、年龄、内脏脂肪、职业.....



- 发现隐藏在数据和答案中的规则(数量关系)并进行预测
 - 与统计学科有着高度一致的目标！
 - 机器学习更关注预测

问题2：统计学专业开设机器学习课程的原因？

- **理解2：**从人工智能的发展：看机器学习与统计学科目标的一致性
 - 人工智能的发展：**符号主义人工智能** (symbolic AI)、**机器学习**、**深度学习**，等
 - **深度学习**：2006年提出，从数据中学习“数据表示”
 - 强调基于数据集，通过众多连续的**神经网络**层，过滤和提取数据中的服务于**预测**的重要特征



- 计算机视觉，大语言模型，等

问题3：经典统计方法与机器学习算法的互补性

- 经典统计方法能达成既定目标吗？经典统计方法的**优势和不足**，通过**案例**讨论

- **案例一**：基于空气质量监测数据，分析CO对PM2.5浓度的**影响效应**（数量关系）；对PM2.5进行**预测**

- PM2.5的主要来源：自然源(如风沙尘土、森林火灾)和人为源(如工业排放、机动车尾气、燃煤燃烧等)
- PM2.5是PM10的组成部分，PM10中的扬尘源(如建筑工地、道路扬尘)会携带大量PM2.5

- 问：采用**什么分析方法**？

$$PM2.5 = \beta_0 + \beta_1 CO + \varepsilon$$

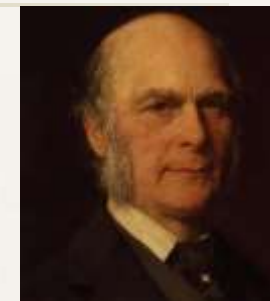
- 问：**为什么**采用此方法？——方法的简要回顾

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3
0	2014-01-01	81.0	良	45.0	111.0	28.0	1.5	62.0	52.0
1	2014-01-02	145.0	轻度污染	111.0	168.0	69.0	3.4	93.0	14.0
2	2014-01-03	74.0	良	47.0	98.0	29.0	1.3	52.0	56.0
3	2014-01-04	149.0	轻度污染	114.0	147.0	40.0	2.8	75.0	14.0
4	2014-01-05	119.0	轻度污染	91.0	117.0	36.0	2.3	67.0	44.0

一般线性回归模型的简要回顾

一般线性回归模型

- 回归的起源：英国生物学家弗朗西斯·高尔顿研究人类遗传进化，1855年发表论文《遗传的身高向平均数方向的回归》
 - 搜集1078对父亲身高(x)及其成儿子身高(y)数据，分析父子身高间的数量关系



回归线：从局部平均到函数拟合

- 局部平均：回归线可由众多局部点的均值点平滑得到
- 函数拟合：假定回归线为直线，有：

$$E(y|x) = \beta_0 + \beta_1 x$$



一元线性回归方程

$$y_i = E(y|x_i) + \varepsilon_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$



• β_0, β_1 分别为截距和斜率

一元线性回归模型

可将 ε 视为标准化值： $E(\varepsilon) = 0$

• β_0, β_1 为模型参数(待估)



依某种策略估计出参数

- 高尔顿研究发现： $\hat{y} = 0.8567 + 0.516x$

• 问：0.516的实际意义？

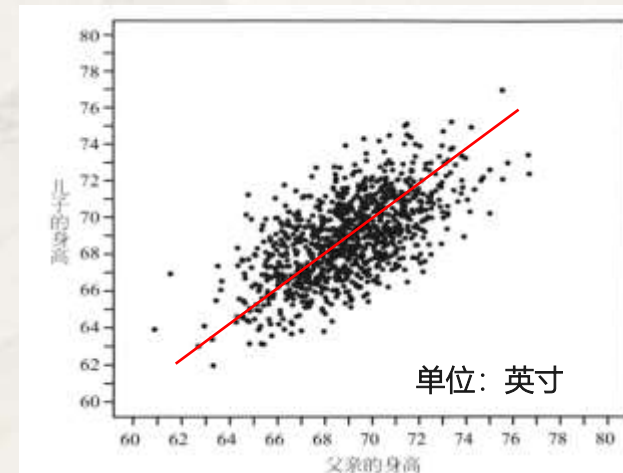
$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

• 估计的一元线性回归方程

• $\hat{y}$ 为预测值或拟合值

• 与2维平面中的直线相对应：回归直线

• 预测值落在回归直线上



$$y_i = E(y|x_i) + \varepsilon_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

$$y_j = E(y|x_i + \Delta x) + \varepsilon_j = \beta_0 + \beta_1 (x_i + \Delta x) + \varepsilon_j$$

$$(y_j - y_i) = \Delta E(y) + \Delta \varepsilon = \beta_1 \Delta x + \Delta \varepsilon$$

• 其他因素不变下， x 的单位变动引起的 y 均值的变动为 β_1

一般线性回归模型的简要回顾

• 案例一的分析：建立一元线性回归模型

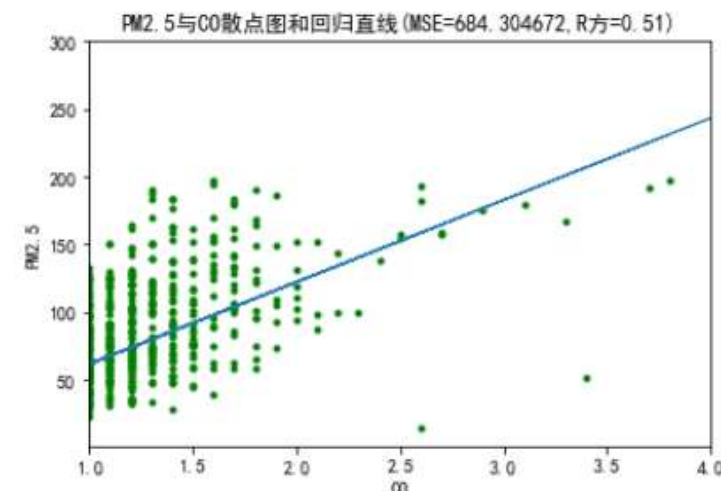
chapter3-1.ipynb

```

1 data = pd.read_excel('北京市空气质量数据.xlsx')
2 data = data.replace(0, np.NaN)
3 data = data.dropna()
4 data = data.loc[(data['PM2.5'] <= 200) & (data['SO2'] <= 20)]
5 X = data[['CO']]
6 y = data['PM2.5']
7 modelLR = LM.LinearRegression()
8 modelLR.fit(X, y)
9 print('一元线性回归模型的截距项:%f'%modelLR.intercept_)
10 print('一元线性回归模型的回归系数:', modelLR.coef_)
11 plt.figure(dpi=150)
12 plt.scatter(data['CO'], data['PM2.5'], c='green', marker='.')
13 plt.plot(data['CO'], modelLR.predict(X), linewidth=0.8)
14 plt.title('PM2.5与CO散点图和回归直线 (MSE=%f, R方=%.2f)'%((sum((y-modelLR.predict(X))**2)/(len(y)-1-1)), modelLR.score(X, y)))
15 plt.xlabel('CO')
16 plt.ylabel('PM2.5')
17 plt.xlim(xmax=4, xmin=1)
18 plt.ylim(ymax=300, ymin=1)

```

一元线性回归模型的截距项: 0.955268
一元线性回归模型的回归系数: [60.58696023]



• 估计的回归方程: $PM2.5 = 0.96 + 60.59CO$

• 问:

- 如何解释模型的实际含义?
- 模型是否需要改进?

一般线性回归模型的简要回顾

- 从一元线性回归模型到多元线性回归模型

$$y = \beta_0 + \beta_1 x_1 + \varepsilon$$



$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

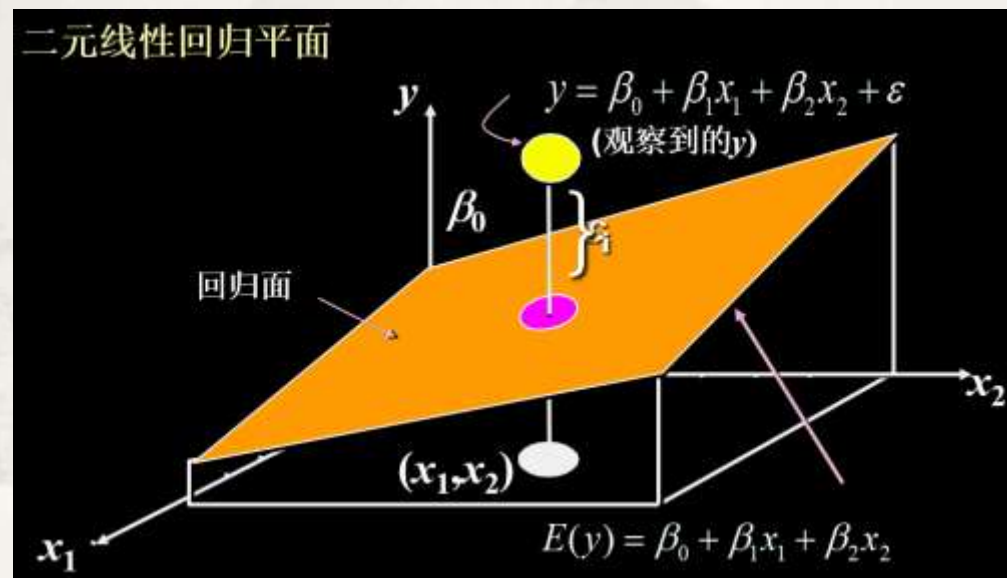


$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon = f(\mathbf{x}) + \varepsilon$$



$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p$$

- 几何理解：预测值落在回归平面上



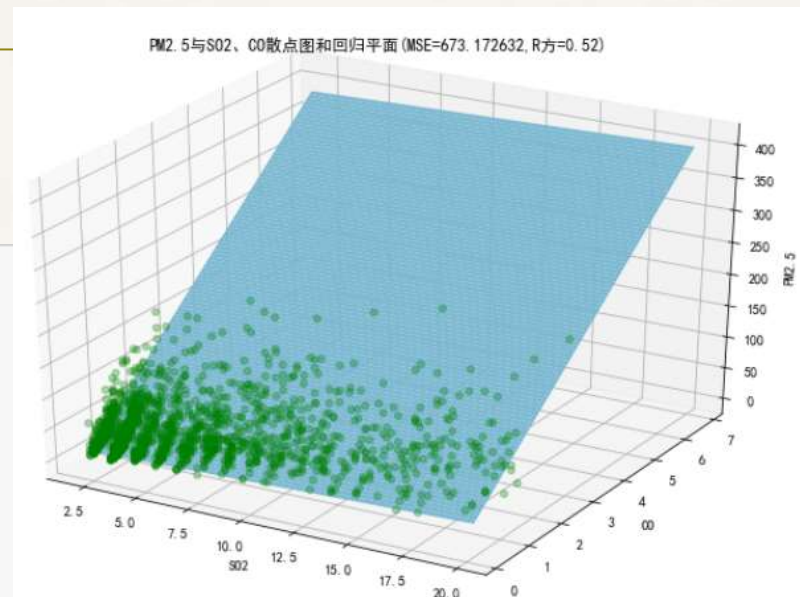
• 案例一的分析：考虑引入SO2

• 建立二元线性回归模型 chapter3-1.ipynb

```

1 X=data[['SO2','CO']]
2 y=data['PM2.5']
3 modelLR=LM.LinearRegression()
4 modelLR.fit(X,y)
5 print('多元线性回归模型的截距项:%f'%modelLR.intercept_)
6 print('多元线性回归模型的回归系数:',modelLR.coef_)
7 ax = Axes3D(plt.figure(figsize=(9, 6), dpi=150))
8 ax.scatter(data[['SO2']],data[['CO']],y,c='green',marker='.',s=100,alpha=0.3)
9 ax.set_xlabel('SO2')
10 ax.set_ylabel('CO')
11 ax.set_zlabel('PM2.5')
12 ax.set_title('PM2.5与SO2、CO散点图和回归平面 (MSE=%f,R方=%.2f)'%((sum((y-modelLR.predict(X))**2)/(len(y)-2-1)),modelLR.score(X,y)))
13 X01 = np.linspace(data[['SO2']].values.min(), data[['SO2']].values.max(), 100)
14 X02 = np.linspace(data[['CO']].values.min(), data[['CO']].values.max(), 100)
15 X01,X02 = np.meshgrid(X01, X02)
16 #Y=modelLR.intercept_+modelLR.coef_[0]*X01+modelLR.coef_[1]*X02
17 X0 = np.hstack((X01.reshape(10000,1),X02.reshape(10000,1)))
18 Y0hat = modelLR.predict(X0).reshape(X01.shape)
19 ax.plot_surface(X01,X02,Y0hat,rstride=1,cstride=1,color='skyblue')

```



多元回归模型的截距项:-1.249076

多元回归模型的回归系数: [0.85972339 56.85521851]

• 问:

- 如何解释模型的实际含义? $PM2.5 = -1.25 + 0.86SO2 + 56.86CO$
- 与一元模型对比, CO的系数为什么变化了? $PM2.5 = 0.96 + 60.59CO$
 - 图示说明, 得到什么启示?
- 模型是否需要改进? 还需要关注哪些方面?

- 案例一的分析：引入其他污染物浓度变量，建立多元线性回归模型并进行检验

```
data.rename(columns={'PM2.5': 'PM'}, inplace=True)
model=STAT.ols('PM~PM10+SO2+CO+NO2+O3',data=data).fit()
model.summary()
```

- 问：模型是否需要改进？

$$PM2.5 = \beta_0 + \beta_1 PM10 + \beta_2 CO + \beta_3 NO_2 + \beta_4 O_3 + \varepsilon$$

```
data[['PM', 'PM10', 'SO2', 'CO', 'NO2', 'O3']] = data[['PM', 'PM10', 'SO2', 'CO', 'NO2', 'O3']].apply(preprocessing.scale)
model=STAT.ols('PM~PM10+CO+NO2+O3',data=data).fit()
model.summary()
```

Dep. Variable:	PM	R-squared:	0.864
Model:	OLS	Adj. R-squared:	0.864
Method:	Least Squares	F-statistic:	2655.
Date:	Thu, 10 Jul 2025	Prob (F-statistic):	0.00
Time:	14:33:01	Log-Likelihood:	-883.46
No. Observations:	2096	AIC:	1779.
Df Residuals:	2090	BIC:	1813.
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	3.99e-17	0.008	4.95e-15	1.000	-0.016	0.016
PM10	0.4912	0.014	35.289	0.000	0.464	0.519
SO2	-0.0241	0.011	-2.238	0.025	-0.045	-0.003
CO	0.4428	0.015	29.110	0.000	0.413	0.473
NO2	0.1035	0.016	6.551	0.000	0.073	0.134
O3	0.0967	0.009	10.472	0.000	0.079	0.115

	coef	std err	t	P> t	[0.025	0.975]
Intercept	2.602e-17	0.008	3.22e-15	1.000	-0.016	0.016
PM10	0.4888	0.014	35.188	0.000	0.462	0.516
CO	0.4348	0.015	29.385	0.000	0.406	0.464
NO2	0.0978	0.016	6.266	0.000	0.067	0.128
O3	0.0990	0.009	10.774	0.000	0.081	0.117

- 问：
 - 进行数据标准化处理的目的？
 - 针对既定目标，其默认满足了哪些前提条件？
 - 各种假定，有严格的数理支撑

问题3：经典统计方法与机器学习算法的互补性

- 经典统计方法能达成既定目标吗？经典统计方法的**优势和不足**，通过**案例讨论**

- **案例一**：基于空气质量监测数据，分析CO对PM2.5浓度的**影响效应**(数量关系)；对PM2.5进行**预测**

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3
0	2014-01-01	81.0	良	45.0	111.0	28.0	1.5	62.0	52.0
1	2014-01-02	145.0	轻度污染	111.0	168.0	69.0	3.4	93.0	14.0
2	2014-01-03	74.0	良	47.0	98.0	29.0	1.3	52.0	56.0
3	2014-01-04	144.0	轻度污染	114.0	147.0	40.0	2.8	75.0	14.0
4	2014-01-05	119.0	轻度污染	91.0	117.0	36.0	2.3	67.0	44.0

- **数据预测**：基于已有数据集，归纳出输入变量和输出变量之间的数量关系

- 数据集中**输入变量**和**输出变量**的值均是已知
- 目的:在数量关系具有普适性和未来不变的假设下，预测新数据的输出变量取值

- **回归预测和分类预测(二分类预测和多分类预测)**

- 例如：PM2.5浓度的回归预测；空气质量等级的分类预测

- **问**：

- 可利用一般线性回归模型实现**数据预测**？
- 针对预测问题需关注什么？

→AQI是空气质量指数（Air Quality Index）的简称，是新环境空气质量标准用于描述空气质量状况的指标。

空气质量指数 (AQI)

AQI 值	AQI 分类	AQI 颜色
0 - 50	优	绿色
51 - 100	良	黄色
101 - 150	轻度污染	橙色
151 - 200	中度污染	红色
201 - 300	重度污染	紫色
301 - 500	严重污染	褐红色

→按照AQI的数值大小将空气质量状况分成了优、良、轻度污染、中度污染、重污染、严重污染六个等级。

问题3：经典统计方法与机器学习算法的互补性

- 经典统计方法能达成既定目标吗？经典统计方法的**优势和不足**，通过**案例讨论**

- **案例一**：基于空气质量监测数据，分析CO对PM2.5浓度的**影响效应**（数量关系）；对PM2.5进行**预测**

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3
0	2014-01-01	81.0	良	45.0	111.0	28.0	1.5	62.0	52.0
1	2014-01-02	145.0	轻度污染	111.0	168.0	69.0	3.4	93.0	14.0
2	2014-01-03	74.0	良	47.0	98.0	29.0	1.3	52.0	56.0
3	2014-01-04	149.0	轻度污染	114.0	147.0	40.0	2.8	75.0	14.0
4	2014-01-05	119.0	轻度污染	91.0	117.0	36.0	2.3	67.0	44.0

- 预测性能的评价
 - 均方误差**MSE**: $\frac{1}{N-p-1} \sum_{i=1}^N e_i^2$ $e_i = y_i - \hat{y}_i$

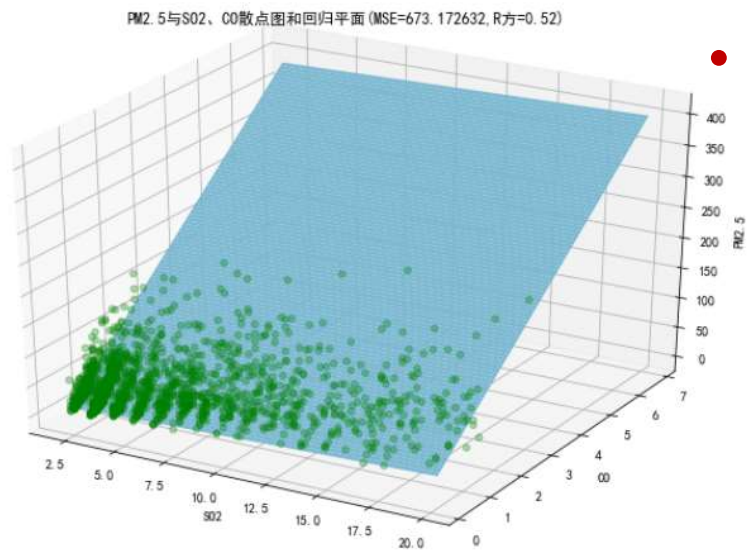
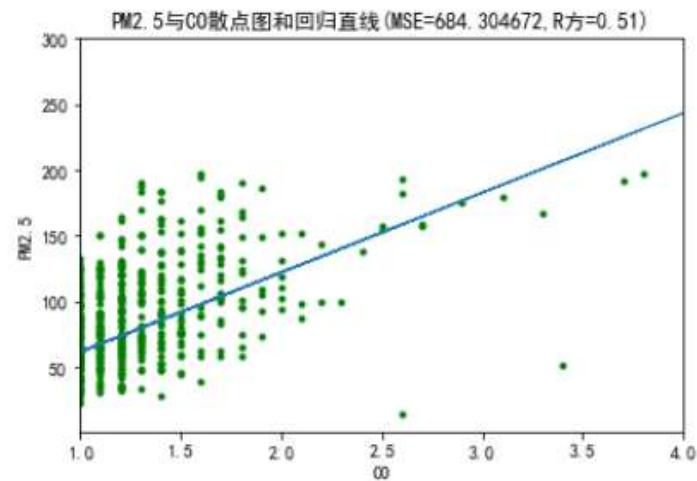
$$\Rightarrow \frac{1}{N} \sum_{i=1}^N e_i^2$$

- MSE越小越好，预测误差越小
- 平均绝对误差**MAE**
- 拟合优度：**R方**，越接近1越好

- e_i 称为残差， N, p 分别为样本量和输入变量的个数
- **问**:
 - MSE中对残差进行平方的作用？
 - MSE中 p 的作用？
 - MSE的不足？

问题3：经典统计方法与机器学习算法的互补性

一元线性回归模型的截距项:0.955268
一元线性回归模型的回归系数:[60.58696023]



问：

- 多元线性回归的预测性能更优一些，为什么？
 - 模型复杂度
 - 有什么启示？
- 一般线性回归模型的整体建模思路是什么？
 - 验证式分析，基于线性的假设

```
1 data = pd.read_excel('北京市空气质量数据.xlsx')
2 data = data.replace(0, np.NaN)
3 data = data.dropna()
4 X = data[['PM10', 'CO', 'NO2', 'O3']]
5 y = data['PM2.5']
6 modelLR = LM.LinearRegression()
7 modelLR.fit(X, y)
8 print('多元线性回归模型: \nR方=%f; \nMSE=%f; \nMAE=%f' % (modelLR.score(X, y), mean_squared_error(y, modelLR.predict(X)),
9                                                         mean_absolute_error(y, modelLR.predict(X))))
```

多元线性回归模型：

R方=0.863645;
MSE=483.959314;
MAE=14.329803

$$PM2.5 = \beta_0 + \beta_1 PM10 + \beta_2 CO + \beta_3 NO_2 + \beta_4 O_3 + \varepsilon$$

问题3：经典统计方法与机器学习算法的互补性

■ 学习机器学习算法的必要性？通过回归预测案例讨论

■ 案例二：共享单车日租赁量(Casual/Registered/count)的预测：回归预测

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

- Season – 1:spring 2:summer 3:fall 4:winter
- Holiday – whether the day is considered a holiday
- Workingday: whether the day is neither a weekend nor holiday
- Weather -
 - 1: Clear, Few clouds, Partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog

- Temp - temperature in Celsius
- Atemp - "feels like" temperature in Celsius
- Humidity - relative humidity
- Windspeed - wind speed
- Casual – number of non-registered user rentals initiated
- Registered - number of registered user rentals initiated
- count - number of total rentals

问题3：经典统计方法与机器学习算法的互补性

■ 学习机器学习算法的必要性？通过回归预测案例讨论

■ 案例二：共享单车日租赁量(Casual/Registered/count)的预测：回归预测

■ 对休闲骑行和常规骑行日租赁量分别预测(业务场景)

■ 采用一般线性回归模型

$$casual = \beta_0 + \beta_1 temp + \beta_2 atemp + \beta_3 humidity + \beta_4 windspeed$$

$$registered = \beta_0 + \beta_1 temp + \beta_2 atemp + \beta_3 humidity + \beta_4 windspeed$$

```
X = data.iloc[:, [4, 5, 6, 7]]
modellr = LM.LinearRegression()
for Y in ['casual', 'registered']:
    y = data[Y]
    modellr.fit(X, y)
    print('\n对%s的预测: R方=%f;MAE=%f'%(Y, modellr.score(X, y), mean_absolute_error(y, modellr.predict(X))))
```

对casual的预测: R方=0.324223;MAE=26.466792

对registered的预测: R方=0.168645;MAE=100.595709

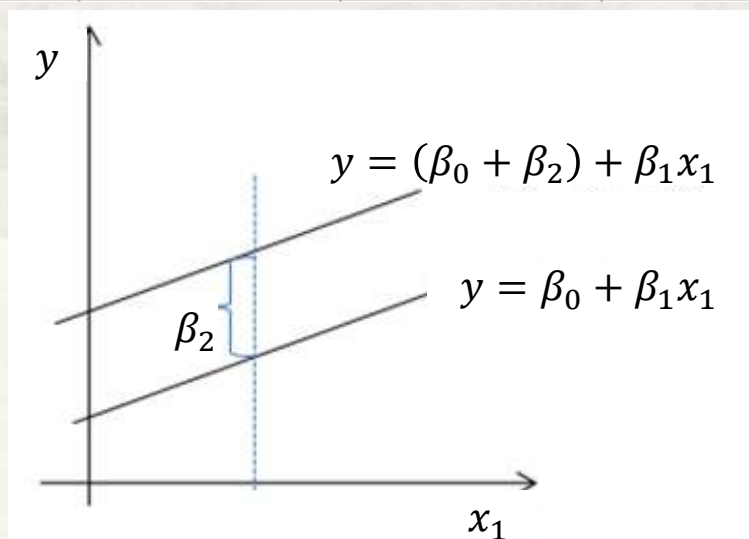
• 问题与思考:

- 为什么预测误差较大?
- 如何改进?

	casual	registered	count
count	9550.000000	9550.000000	9550.000000
mean	36.957906	159.184817	196.142723
std	50.270002	151.809675	181.981750
min	0.000000	0.000000	1.000000
25%	4.000000	41.000000	47.000000
50%	18.000000	122.000000	150.000000
75%	50.000000	227.000000	289.000000
max	362.000000	886.000000	977.000000

- 引入分类型变量：分类型变量的处理
 - 例：

	A	B	C	D
	月工资收入 (元) y	工作年限 (年) x_1	性别 x_2	x_2
1			男	1
2	2900	2	女	0
3	3000	6	女	0
4	4800	8	男	1
5	1800	3	女	0
6	2900	2	男	1
7	4900	7	男	1
8	4200	9	女	0
9	4800	8	女	0
10	4400	4	男	1
11	4500	6	男	1



- 采用带虚拟变量的线性回归分析
- 虚拟变量的取只有0和1两个值，表示“是”和“不是”
- 对有 K 个类别的分类型变量，只能有 $K - 1$ 个虚拟变量参与建模
 - 指定一个参照类：这里女性为参照类

$$E(y) = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$\text{女}(x_2=0): E(y|\text{女性}) = \beta_0 + \beta_1 x_1$$

$$\text{男}(x_2=1): E(y|\text{男性}) = (\beta_0 + \beta_2) + \beta_1 x_1$$

- 问：
 - 如何解释模型的实际含义？

- 案例二的分析：建立一般线性回归模型
 - 对休闲骑行和常规骑行日租赁量分别预测(业务场景)
 - 调整模型：将分类变量(转为虚拟变量形式后)引入回归模型

season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count	season_2	season_3	season_4	weather_2	weather_3
1	0	0	2	9.84	12.880	75.0	6.0032	0	1	1	0	0	0	1	0
1	0	0	1	15.58	19.695	76.0	16.9979	12	24	36	0	0	0	0	0
1	0	0	1	14.76	16.665	81.0	19.0012	26	30	56	0	0	0	0	0
1	0	0	1	17.22	21.210	77.0	19.0012	29	55	84	0	0	0	0	0
1	0	0	2	18.86	22.725	72.0	19.9995	47	47	94	0	0	0	1	0

- 结果解读：较前个模型预测误差有所降低

调整后的线性回归模型

对casual的预测：R方=0.452398;MAE=25.435649
对registered的预测：R方=0.215822;MAE=97.870740

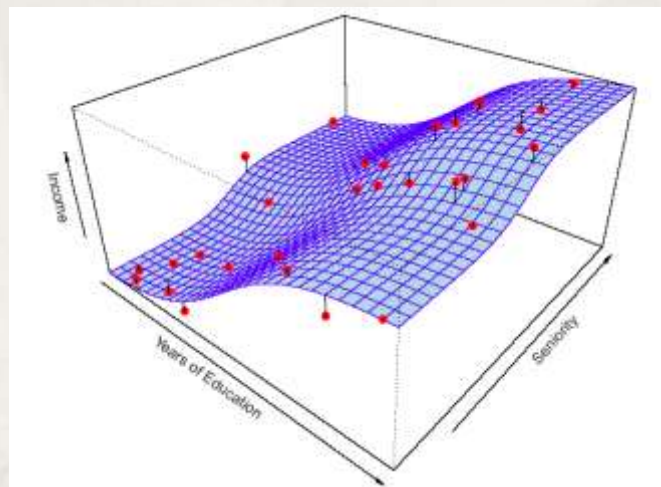
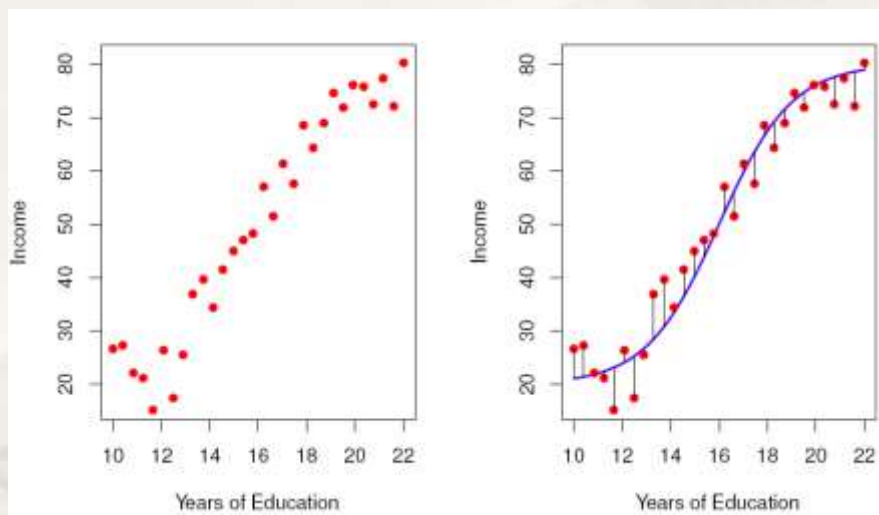
前个模型

对casual的预测：R方=0.324223;MAE=26.466792
对registered的预测：R方=0.168645;MAE=100.595709

- 问：
 - 为什么预测误差有所降低？
 - 为什么预测结果仍不理想？再看基于线性回归模型的数据预测

问题3：经典统计方法与机器学习算法的互补性

- 学习机器学习算法的**必要性**？通过**回归预测案例**讨论



- 预测的核心任务：估计 f

$$Y = f(X) + \epsilon.$$

- f 是 $\mathbf{X}$ 对 y 的系统性信息（未知）
- $E(\epsilon) = 0$ 独立于 $\mathbf{X}$

问题3：经典统计方法与机器学习算法的互补性

- 学习机器学习算法的**必要性**？通过**回归预测案例**讨论
 - 经典统计方法**如何估计** f ？

$$Y = f(X) + \epsilon.$$

$$\hat{f}(\mathbf{X}) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

- 对任意观测 $(\mathbf{X}, y)$ 有：

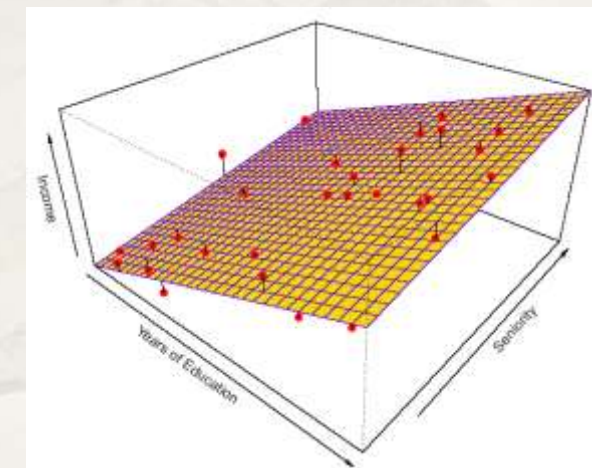
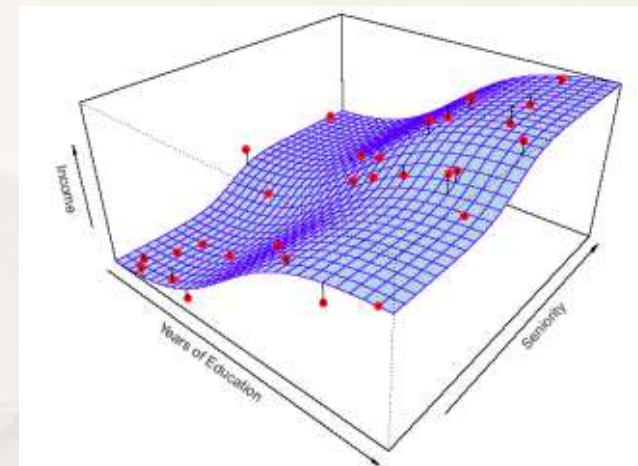
$$Y \approx \hat{f}(X)$$

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p.$$

$$\text{income} \approx \beta_0 + \beta_1 \times \text{education} + \beta_2 \times \text{seniority}.$$

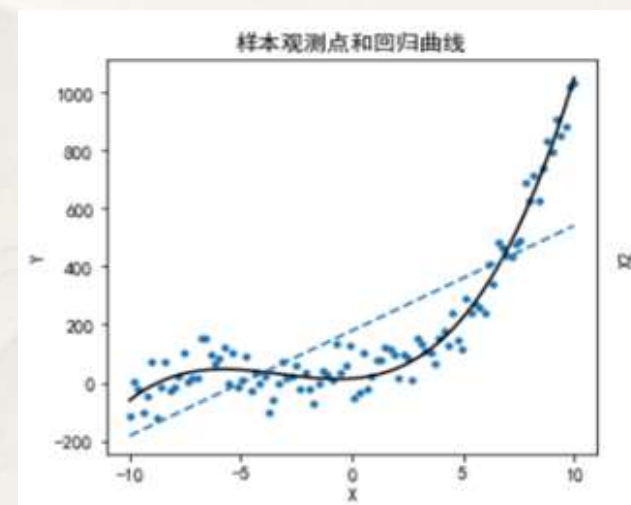
- 归结为对**未知参数** $\beta_1, \dots, \beta_p$ 的估计问题！
 - 例如：最小二乘估计

$$\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \cdots + \hat{\beta}_p X_p$$



问题3：经典统计方法与机器学习算法的互补性

- 学习机器学习算法的**必要性**？通过**回归预测案例**讨论
 - **问**：案例二采用这种方式**恰当吗？为什么**？
 - **结论**：机器学习的必要性在于：从平线(面)到曲线(面)
 - **非线性回归预测**和**分类预测**(后续讲)
 - 在**提高预测精度**方面可以做得更好
 - 有效解决**非线性回归**问题
 - **问**：如何看待曲线(曲面)
 - 曲面由许多局部平面平滑连接的结果
 - 是高维 G 空间 $\mathbb{R}^G$ 中平面在 I ($G > I$) 维空间 $\mathbb{R}^I$ 中的一种呈现
- 机器学习：从统计的**模型驱动**→通过各种**算法**实现**数据驱动**的**归纳式探索**
 - **问**：什么是算法？

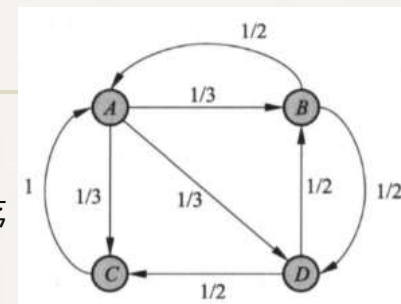


■ 算法：一般指计算机中解决某个问题的一系列清晰指令的集合。如：

■ 排序算法 (交换排序【冒泡法、快速排序】、插入排序【直接法、希尔排序】、选择排序等)

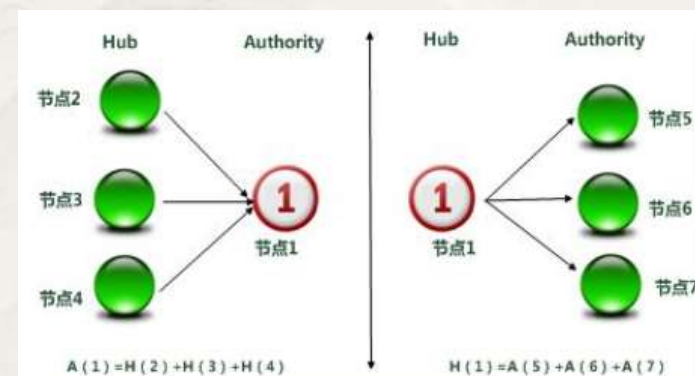
■ PageRank算法

- PageRank算法是图的链接分析(link analysis)的代表性算法，提出时服务于Google搜索引擎排序
- 定义在有向图上的随机游走过程，迭代算法
 - 如果指向A网页的超链接越多，随机跳转到A网页的概率就越高，A网页的PageRank值就越高，A网页越重要
 - 如果指向A网页的多个网页的PageRank值越高，A网页的PageRank值就越高，A网页越重要
- PageRank可以定义在任意有向图上，后来被应用到社会影响力分析、文本摘要等多个方面



■ HITS (Hypertext-Induced Topic Search) 算法

- 利用网页内容权威值(Authority)和链接枢纽值(Hub)对网页评估(学术文献评价)
- 迭代算法
 - 权威值：所有导入链接所在页面的枢纽值之和；
 - 枢纽值：页面上所有链接指向页面的权威值之和



■ 协同过滤算法

- 推荐算法，基于用户的协同过滤和基于商品的协同过滤，等

■ 机器学习算法一般指以什么策略(怎么做)在数据中找规则

- 涉及数学和优化方法等
- 本课程只要求掌握算法的设计思路

课下温习的主要内容

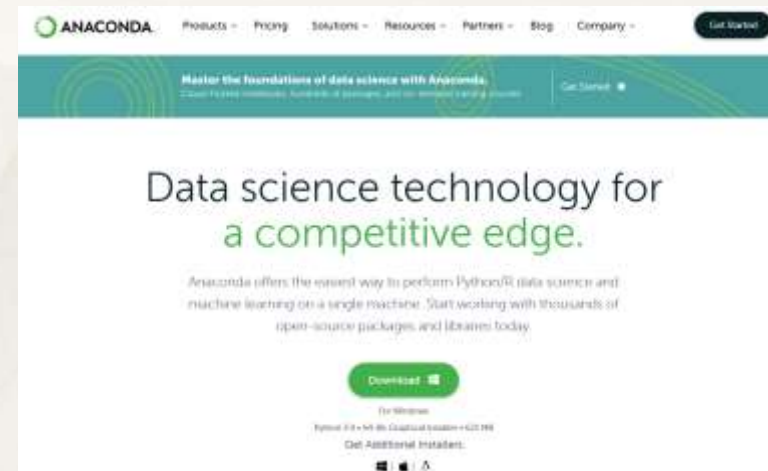
- Python基础
 - Python的集成开发环境：Anaconda
 - Python第三方包的引用
 - NumPy使用示例
 - Pandas使用示例
 - NumPy和Pandas的综合应用
 - Matplotlib的综合应用
 - Seaborn的综合应用

Python：机器学习的首选工具

- Python是一款**面向对象**的解释型计算机语言。开源、代码可读性强，可实现高效开发等是Python的重要特征
 - 面向对象的程序设计(Object Oriented Programming, OOP)是相对面向过程的程序设计而言的
 - **OOP**采用“封装”的思想，将具有一定独立性和通用性的处理过程和变量(数据)，封装在“对象”中
 - 变量称为对象的“**属性**”，变量值对应属性值(有具体变量值的对象称为“对象实例”)
 - 处理过程称为对象的“**方法**” (本质是函数)
 - 多个具有内在联系的对象可进一步封装在“**类**”中
- Python在机器学习领域获得广泛使用的原因：
 - 简明易用，严谨专业
 - 良好的开发社区生态
 - 丰富的第三方程序包
 - **NumPy**
 - **Pandas**
 - **SciPy**
 - **Matplotlib**
 - **Seaborn**
 - **Scikit-learn** (skLearn)

Python的集成开发环境：Anaconda

- Anaconda：兼容Linux、Windows和Mac OS X. 环境，支持Python 2.X和Python 3.X，且可方便快捷完成机器学习和数据科学任务的开源IDE
 - 通常将Anaconda视为Python的发行版Anaconda®
 - Anaconda的官方下载地址为：<https://www.anaconda.com/>
- Anaconda Prompt的使用



```
Anaconda Prompt (Anaconda3) - python

(base) C:\Users\xuewe>python
Python 3.7.4 (default, Aug 9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello Python!")
Hello Python!
>>> .
```

Python的集成开发环境：Anaconda

- Spyder的使用：Python程序的扩展名为.py

The screenshot displays the Spyder Python IDE interface. The main editor window shows a Python script named `MyPython1.py` with the following code:

```
1 # -*- coding: utf-8 -*-
2 """
3 author:xuwei
4
5 读数据文件:
6     1、读Excel文件
7
8 """
9
10 import pandas as pd
11 df=pd.read_excel("北京市空气质量数据.xlsx")#这个会直接默认读取到这个Excel的最后一个表
12 data=df.head()#默认读取前5行的数据
13 print('获取到所有的值:\n{0}'.format(data))#格式化输出
14
15
```

The Variable explorer on the right shows two variables:

Name	Type	Size	Value
data	DataFrame	(5, 9)	Column names: 日期, AQI, 质量等级, PM2.5, PM10, SO2, CO, NO2, O3
df	DataFrame	(2155, 9)	Column names: 日期, AQI, 质量等级, PM2.5, PM10, SO2, CO, NO2, O3

The IPython console at the bottom shows the execution of the script:

```
Python 3.7.4 (default, Aug 9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.8.0 -- An enhanced Interactive Python.

In [1]: runfile('C:/Users/xuwei/Desktop/MyPython1.py', wdir='C:/Users/xuwei/Desktop')
获取到所有的值:
   日期 AQI 质量等级 PM2.5 PM10 SO2 CO NO2 O3
0 2014-01-01    81    良    45   111   28  1.5   62  52
1 2014-01-02   145  轻度污染   111   168   69  3.4   93  14
2 2014-01-03    74    良    47    98   29  1.3   52  56
3 2014-01-04   149  轻度污染   114   147   40  2.8   75  14
4 2014-01-05   119  轻度污染    91   117   36  2.3   67  44

In [2]: %hist
runfile('C:/Users/xuwei/Desktop/MyPython1.py', wdir='C:/Users/xuwei/Desktop')
%hist

In [3]: |
```

- Jupyter Notebook的使用: 1984年提出的文字表达化编程形式的具体体现
 - 是一个基于网页的交互式笔记本；支持多种语言，Python程序的扩展名为.ipynb



```
In [1]: 1 import pandas as pd
        2 df=pd.read_excel('北京市空气质量数据.xlsx')
        3 data=df.head()
        4 print('获取到所有的值:\n{0}'.format(data))
```

获取到所有的值：

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3
0	2014-01-01	81	良	45	111	28	1.5	62	52
1	2014-01-02	145	轻度污染	111	168	69	3.4	93	14
2	2014-01-03	74	良	47	98	29	1.3	52	56
3	2014-01-04	149	轻度污染	114	147	40	2.8	75	14
4	2014-01-05	119	轻度污染	91	117	36	2.3	67	44

说明：1、利用Pandas的read_excel将一个关于空气质量监测数据的Excel数据文件读到数据框 (DataFrame)中。2、head默认读取前五行观测数据。3、print输出所读取的前五行观测数据。

Python第三方包的引用

- 第三方包以模块(Module, 文件扩展名为.py)方式, 将可实现各种功能的程序代码(变量、函数)“打包”在一起
 - 包由多个相关模块组成, 一般以文件目录的形式组织(目录名为包名)
- 引用第三方包中的模块的基本函数: import函数
 - import 包名或模块名: 导入指定包或模块
 - import 包名.模块名: 导入指定包中的指定模块
 - from 模块名 import 函数名: 导入指定模块中的指定函数
 - from 模块名 import 函数名1, 函数名2, ...: 导入指定模块中的若干个指定函数
 - from 模块名 import *: 导入指定模块中的所有函数
 - 可增加: as 别名
 - import numpy as np, 表示导入numpy并指定别名为np
 - 指定别名可以有效避免不同模块有相同函数名的问题
 - 之后可在自己编写的Python程序直接调用已导入模块中的函数, 通过代码重用(重复使用)的方式快速实现某种特定功能

- NumPy (Numerical Python) 使用示例：最常用的 Python 包。特点：
 - 第一，以N维（ $N = 1$ 或2比较常见）数组对象(ndarray)形式组织数据集(数据集中各变量的类型相同)
 - 1维数组对应向量，2维数据对应矩阵
 - 数据访问简单灵活，仅需通过指定位置编号（也称索引，从0开始编号）访问相应行列位置上的元素

```

1 import numpy as np
2 data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
3 print('Numpy的1维数组:\n{0}'.format(data))
4 print('数据类型:%s'%data.dtype)
5 print('1维数组中各元素扩大10倍:\n{0}'.format(data*10))
6 print('访问第2个元素: {0}'.format(data[1]))
7 data = np.array([[1, 3, 5, 7, 9], [2, 4, 6, 8, 10]])
8 print('Numpy的2维数组:\n{0}'.format(data))
9 print('访问2维数组中第1行第2列元素: {0}'.format(data[0, 1]))
10 print('访问2维数组中第1行第2至4列元素: {0}'.format(data[0, 1:4]))
11 print('访问2维数组中第1行上的所有元素: {0}'.format(data[0, :]))
    
```

Chapter2-1.ipynb

Numpy的1维数组:
 [1 2 3 4 5 6 7 8 9]
 数据类型:int32
 1维数组中各元素扩大10倍:
 [10 20 30 40 50 60 70 80 90]
 访问第2个元素: 2
 Numpy的2维数组:
 [[1 3 5 7 9]
 [2 4 6 8 10]]
 访问2维数组中第1行第2列元素: 3
 访问2维数组中第1行第2至4列元素: [3 5 7]
 访问2维数组中第1行上的所有元素: [1 3 5 7 9]

- 列表：Python重要的数据组织形式，NumPy数组数据的重要来源
 - NumPy数组中元素应有相同的数据类型，列表可以不同

```

1 data = [[1, 2, 3, 4, 5, 6, 7, 8, 9], ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']]
2 print('data是Python的列表(list):\n{0}'.format(data))
3 MyArray1 = np.array(data)
4 print('MyArray1是Numpy的N维数组:\n%s\nMyarray1的形状:%s'%(MyArray1, MyArray1.shape))
    
```

data是Python的列表(list):
 [[1, 2, 3, 4, 5, 6, 7, 8, 9], ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']]
 MyArray1是Numpy的N维数组:
 [['1' '2' '3' '4' '5' '6' '7' '8' '9']
 ['A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I']]
 Myarray1的形状:(2, 9)

- NumPy (Numerical Python) 使用示例：最常用的 Python 包。特点：
 - **第二**，拥有丰富的数学运算和统计函数

```
1 MyArray2 = np.arange(10)
2 print('MyArray2:\n{0}'.format(MyArray2))
3 print('MyArray2的基本描述统计量:\n均值: %f, 标准差: %f, 总和: %f, 最大值: %f' % (MyArray2.mean(), MyArray2.std(), MyArray2.sum(), MyArray2.max))
4 print('MyArray2的累计和: {0}'.format(MyArray2.cumsum()))
5 print('MyArray2开平方: {0}'.format(np.sqrt(MyArray2)))
6 np.random.seed(123)
7 MyArray3 = np.random.randn(10)
8 print('MyArray3:\n{0}'.format(MyArray3))
9 print('MyArray3排序结果:\n{0}'.format(np.sort(MyArray3)))
10 print('MyArray3四舍五入到最近整数:\n{0}'.format(np rint(MyArray3)))
11 print('MyArray3各元素的正负号: {0}'.format(np.sign(MyArray3)))
12 print('MyArray3各元素非负数的显示“正”，负数显示“负”:\n{0}'.format(np.where(MyArray3>0, '正', '负')))
13 print('MyArray2+MyArray3的结果:\n{0}'.format(MyArray2+MyArray3))
```

```
MyArray2:
[0 1 2 3 4 5 6 7 8 9]
```

```
MyArray3四舍五入到最近整数:
[-1.  1.  0. -2. -1.  2. -2. -0.  1. -1.]
MyArray3各元素的正负号: [-1.  1.  1. -1. -1.  1. -1. -1.  1. -1.]
MyArray3各元素非负数的显示“正”，负数显示“负”：
['负' '正' '正' '负' '负' '正' '负' '负' '正' '负']
```

- 指定种子的目的是确保每次运行代码时生成的随机数可以再现。否则，每次运行代码生成的随机数会不相同
- random.randn()生成服从均值为0标准差为1的标准正态分布的随机数

- NumPy (Numerical Python) 使用示例：最常用的 Python 包。特点：
 - 第二，拥有丰富的数学运算和统计函数

```
1 np.random.seed(123)
2 X = np.floor(np.random.normal(5, 1, (2, 5)))
3 print('X:\n{0}'.format(X))
4 Y = np.eye(5)
5 print('Y:\n{0}'.format(Y))
6 print('X和Y的点积: \n{0}'.format(np.dot(X, Y)))
```

```
X:
[[3.  5.  5.  3.  4.]
 [6.  2.  4.  6.  4.]]
Y:
[[1.  0.  0.  0.  0.]
 [0.  1.  0.  0.  0.]
 [0.  0.  1.  0.  0.]
 [0.  0.  0.  1.  0.]
 [0.  0.  0.  0.  1.]]
X和Y的点积:
[[3.  5.  5.  3.  4.]
 [6.  2.  4.  6.  4.]]
```

- random.normal()函数生成2行5列的2维数组，数组元素服从均值为5标准差为1的正态分布
- 利用floor函数得到距各数组元素最近的最大整数
- eye(5)函数生成5行5列的单位阵Y

```
1 from numpy.linalg import inv, svd, eig, det
2 np.random.seed(123)
3 X = np.random.randn(5, 5)
4 mat = X.T.dot(X)
5 print(mat)
6 print('矩阵mat的逆: \n{0}'.format(inv(mat)))
7 print('矩阵mat的行列式值: \n{0}'.format(det(mat)))
8 print('矩阵mat的特征值和特征向量: \n{0}'.format(eig(mat)))
9 print('对矩阵mat做奇异值分解: \n{0}'.format(svd(mat)))
```

- Pandas(Python data analysis)使用示例：提供了快速便捷组织和处理结构化数据的数据结构和大量功能丰富的函数。特点：
 - 基于NumPy构建：
 - 序列 (Series)：对应1维数组
 - 数据框 (DataFrame)：对应2维表格型数据结构,称数据框是序列的容器
 - Pandas数据框是存储机器学习数据集的常用形式

```
1 import numpy as np
2 import pandas as pd
3 from pandas import Series, DataFrame
4 data = Series([1,2,3,4,5,6,7,8,9], index=['ID1', 'ID2', 'ID3', 'ID4', 'ID5', 'ID6', 'ID7', 'ID8', 'ID9'])
5 print('序列中的值:\n{}'.format(data.values))
6 print('序列中的索引:\n{}'.format(data.index))
7 print('访问序列的第1和第3上的值:\n{}'.format(data[[0,2]]))
8 print('访问序列索引为ID1和ID3上的值:\n{}'.format(data[['ID1', 'ID3']]))
9 print('判断ID1索引是否存在:%s; 判断ID10索引是否存在:%s'%( 'ID1' in data, 'ID10' in data))
```

Chapter2-2.ipynb

- Pandas的索引

```
序列中的值:
[1 2 3 4 5 6 7 8 9]
序列中的索引:
Index(['ID1', 'ID2', 'ID3', 'ID4', 'ID5', 'ID6', 'ID7', 'ID8', 'ID9'], dtype='object')
访问序列的第1和第3上的值:
ID1    1
ID3    3
dtype: int64
访问序列索引为ID1和ID3上的值:
ID1    1
ID3    3
dtype: int64
判断ID1索引是否存在:True; 判断ID10索引是否存在:False
```

• Pandas的数据框

```
1 import pandas as pd
2 data = pd.read_excel('北京市空气质量数据.xlsx')
3 print('date的类型: {}'.format(type(data)))
4 print('数据框的行索引: {}'.format(data.index))
5 print('数据框的列名: {}'.format(data.columns))
6 print('访问AQI和PM2.5所有值: \n{}'.format(data[['AQI', 'PM2.5']]))
7 print('访问第2至3行的AQI和PM2.5: \n{}'.format(data.loc[1:2, ['AQI', 'PM2.5']]))
8 print('访问行索引1至索引2的第2和4列: \n{}'.format(data.iloc[1:3, [1, 3]]))
9 data.info()
```

date的类型: <class 'pandas.core.frame.DataFrame'>

数据框的行索引: RangeIndex(start=0, stop=2155, step=1)

数据框的列名: Index(['日期', 'AQI', '质量等级', 'PM2.5', 'PM10', 'SO2', 'CO', 'NO2', 'O3'], dtype='object')

访问AQI和PM2.5所有值:

	AQI	PM2.5
0	81	45
1	145	111
2	74	47
3	149	114
4	119	91
...	...	...
2150	183	138
2151	175	132
2152	30	7
2153	40	13
2154	73	38

[2155 rows x 2 columns]

访问第2至3行的AQI和PM2.5:

	AQI	PM2.5
1	145	111
2	74	47

访问行索引1至索引2的第2和4列:

	AQI	PM2.5
1	145	111
2	74	47

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2155 entries, 0 to 2154
Data columns (total 9 columns):
日期          2155 non-null datetime64[ns]
AQI           2155 non-null int64
质量等级      2155 non-null object
PM2.5         2155 non-null int64
PM10          2155 non-null int64
SO2           2155 non-null int64
CO            2155 non-null float64
NO2           2155 non-null int64
O3            2155 non-null int64
```

• Pandas的数据加工处理

```
1 import numpy as np
2 import pandas as pd
3 from pandas import Series, DataFrame
4 df1 = DataFrame({'key': ['a', 'd', 'c', 'a', 'b', 'd', 'c'], 'var1': range(7)})
5 df2 = DataFrame({'key': ['a', 'b', 'c', 'c'], 'var2': [0, 1, 2, 2]})
6 df = pd.merge(df1, df2, on='key', how='outer')
7 df.iloc[0, 2] = np.NaN
8 df.iloc[5, 1] = np.NaN
9 print('合并后的数据:\n{}'.format(df))
10 df = df.drop_duplicates()
11 print('删除重复数据行后的数据:\n{}'.format(df))
12 print('判断是否为缺失值:\n{}'.format(df.isnull()))
13 print('判断是否不为缺失值:\n{}'.format(df.notnull()))
14 print('删除缺失值后的数据:\n{}'.format(df.dropna()))
15 fill_value = df[['var1', 'var2']].apply(lambda x: x.mean())
16 print('以均值替换缺失值:\n{}'.format(df.fillna(fill_value)))
```

	key	var1
0	a	0
1	d	1
2	c	2
3	a	3
4	b	4
5	d	5
6	c	6

	key	var2
0	a	0
1	b	1
2	c	2
3	c	2

	key	var1	var2
0	a	0	0.0
1	a	3	0.0
2	d	1	NaN
3	d	5	NaN
4	c	2	2.0
5	c	2	2.0
6	c	6	2.0
7	c	6	2.0
8	b	4	1.0

合并后的数据:

	key	var1	var2
0	a	0.0	NaN
1	a	3.0	0.0
2	d	1.0	NaN
3	d	5.0	NaN
4	c	2.0	2.0
5	c	NaN	2.0
6	c	6.0	2.0
7	c	6.0	2.0
8	b	4.0	1.0

删除重复数据行后的数据:

	key	var1	var2
0	a	0.0	NaN
1	a	3.0	0.0
2	d	1.0	NaN
3	d	5.0	NaN
4	c	2.0	2.0
5	c	NaN	2.0
6	c	6.0	2.0
8	b	4.0	1.0

判断是否为缺失值:

	key	var1	var2
0	False	False	True
1	False	False	False
2	False	False	True
3	False	False	True
4	False	False	False
5	False	True	False
6	False	False	False
8	False	False	False

判断是否不为缺失值:

	key	var1	var2
0	True	True	False
1	True	True	True
2	True	True	False
3	True	True	False
4	True	True	True
5	True	False	True
6	True	True	True
8	True	True	True

删除缺失值后的数据:

	key	var1	var2
1	a	3.0	0.0
4	c	2.0	2.0
6	c	6.0	2.0
8	b	4.0	1.0

以均值替换缺失值:

	key	var1	var2
0	a	0.0	1.4
1	a	3.0	0.0
2	d	1.0	1.4
3	d	5.0	1.4
4	c	2.0	2.0
5	c	3.0	2.0
6	c	6.0	2.0
8	b	4.0	1.0

NumPy和Pandas的综合应用：空气质量监测数据的预处理和基本分析

• 数据预处理

```
1 import numpy as np
2 import pandas as pd
3 data = pd.read_excel('北京市空气质量数据.xlsx')
4 data = data.replace(0, np.NaN)
5 data['年'] = data['日期'].apply(lambda x:x.year)
6 month = data['日期'].apply(lambda x:x.month)
7 quarter_month = {1:'一季度',2:'一季度',3:'一季度',4:'二季度',5:'二季度',6:'二季度',7:'三季度',8:'三季度',9:'三季度',10:'四季度',11:'四季
8 data['季度'] = month.map(lambda x:quarter_month[x])
9 bins=[0, 50, 100, 150, 200, 300, 1000]
10 data['等级'] = pd.cut(data['AQI'], bins, labels=['一级优', '二级良', '三级轻度污染', '四级中度污染', '五级重度污染', '六级严重污染'])
11 data[['日期', 'AQI', '等级', '季度']]
```

Chapter2-3.ipynb

	日期	AQI	等级	季度
0	2014-01-01	81.0	二级良	一季度
1	2014-01-02	145.0	三级轻度污染	一季度
2	2014-01-03	74.0	二级良	一季度
3	2014-01-04	149.0	三级轻度污染	一季度
4	2014-01-05	119.0	三级轻度污染	一季度
...	...	...	...	...
2150	2019-11-22	183.0	四级中度污染	四季度
2151	2019-11-23	175.0	四级中度污染	四季度
2152	2019-11-24	30.0	一级优	四季度
2153	2019-11-25	40.0	一级优	四季度
2154	2019-11-26	73.0	二级良	四季度

- 数据框data中的“日期”为Python的datetime型，专用于存储日期和时间格式变量

NumPy和Pandas的综合应用：空气质量监测数据的预处理和基本分析

基本分析

```
1 print('各季度AQI和PM2.5的均值:\n{0}'.format(data.loc[:,['AQI','PM2.5']].groupby(data['季度']).mean()))
2 print('各季度AQI和PM2.5的描述统计量:\n',data.groupby(data['季度'])['AQI','PM2.5'].apply(lambda x:x.describe()))
3 def top(df,n=10,column='AQI'):
4     return df.sort_values(by=column,ascending=False)[:n]
5 print('空气质量最差的5天:\n',top(data,n=5)[['日期','AQI','PM2.5','等级']])
6 print('各季度空气质量最差的3天:\n',data.groupby(data['季度']).apply(lambda x:top(x,n=3)[['日期','AQI','PM2.5','等级']]))
7 print('各季度空气质量情况:\n',pd.crosstab(data['等级'],data['季度'],margins=True,margins_name='总计',normalize=False))
```

各季度AQI和PM2.5的均值:			各季度AQI和PM2.5的描述统计量:			空气质量最差的5天:				各季度空气质量情况:						
	AQI	PM2.5		AQI	PM2.5	日期	AQI	PM2.5	等级	季度	一季度	三季度	二季度	四季度	总计	
季度			季度			1218	2017-05-04	500.0	NaN	六级严重污染						
一季度	109.327778	77.225926	一季度 count	540.000000	540.000000	723	2015-12-25	485.0	477.0	六级严重污染	一级优	145	96	38	387	
三季度	98.911071	49.528131	mean	109.327778	77.225926	699	2015-12-01	476.0	464.0	六级严重污染	二级良	170	209	240	849	
二季度	109.369004	55.149723	std	80.405408	73.133857	1095	2017-01-01	470.0	454.0	六级严重污染	三级轻度污染	99	164	152	64	479
四季度	109.612403	77.195736	min	26.000000	4.000000	698	2015-11-30	450.0	343.0	六级严重污染	四级中度污染	57	72	96	33	258
			25%	48.000000	24.000000	各季度空气质量最差的3天:				五级重度污染	48	10	14	58	130	
			50%	80.000000	53.000000	日期	AQI	PM2.5	等级	六级严重污染	21	0	2	23	46	
			75%	145.000000	109.250000	季度				总计	540	551	542	516	2149	
			max	470.000000	454.000000	一季度	1095	2017-01-01	470.0	454.0	六级严重污染					
			三季度 count	551.000000	551.000000	45	2014-02-15	428.0	393.0	六级严重污染						
			mean	98.911071	49.528131	55	2014-02-25	403.0	354.0	六级严重污染						
			std	45.484516	35.394897	三季度	186	2014-07-06	252.0	202.0	五级重度污染					
			min	28.000000	3.000000	211	2014-07-31	245.0	195.0	五级重度污染						
			25%	60.000000	23.000000	183	2014-07-03	240.0	190.0	五级重度污染						
			50%	95.000000	41.000000	二季度	1218	2017-05-04	500.0	NaN	六级严重污染					
			75%	130.500000	67.000000	1219	2017-05-05	342.0	181.0	六级严重污染						
			max	252.000000	202.000000	103	2014-04-14	279.0	229.0	五级重度污染						
			二季度 count	542.000000	541.000000	四季度	723	2015-12-25	485.0	477.0	六级严重污染					
			mean	109.369004	55.149723	699	2015-12-01	476.0	464.0	六级严重污染						
			std	49.608042	35.918345	698	2015-11-30	450.0	343.0	六级严重污染						

NumPy和Pandas的综合应用：空气质量监测数据的预处理和基本分析

• 派生虚拟自变量

```
1 pd.get_dummies(data['等级'])
2 data.join(pd.get_dummies(data['等级']))
```

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3	年	季度	等级	一级优	二级良	三级轻度污染	四级中度污染	五级重度污染	六级严重污染
0	2014-01-01	81.0	良	45.0	111.0	28.0	1.5	62.0	52.0	2014	一季度	二级良	0	1	0	0	0	0
1	2014-01-02	145.0	轻度污染	111.0	168.0	69.0	3.4	93.0	14.0	2014	一季度	三级轻度污染	0	0	1	0	0	0
2	2014-01-03	74.0	良	47.0	98.0	29.0	1.3	52.0	56.0	2014	一季度	二级良	0	1	0	0	0	0
3	2014-01-04	149.0	轻度污染	114.0	147.0	40.0	2.8	75.0	14.0	2014	一季度	三级轻度污染	0	0	1	0	0	0
4	2014-01-05	119.0	轻度污染	91.0	117.0	36.0	2.3	67.0	44.0	2014	一季度	三级轻度污染	0	0	1	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2150	2019-11-22	183.0	中度污染	138.0	181.0	9.0	2.4	94.0	5.0	2019	四季度	四级中度污染	0	0	0	1	0	0

NumPy和Pandas的综合应用：空气质量监测数据的预处理和基本分析

- 抽取数据子集

```
1 np.random.seed(123)
2 sampler = np.random.randint(0, len(data), 10)
3 print(sampler)
4 sampler = np.random.permutation(len(data))[:10]
5 print(sampler)
6 data.take(sampler)
7 data.loc[data['质量等级']=='优',:]
```

```
[1346 1122 1766 2154 1147 1593 1761    96    47    73]
[1883   326    43 1627 1750 1440   993 1469 1892  865]
```

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3	年	季度	等级
7	2014-01-08	27.0	优	15.0	25.0	13.0	0.5	21.0	53.0	2014	一季度	一级优
8	2014-01-09	46.0	优	27.0	46.0	19.0	0.8	35.0	53.0	2014	一季度	一级优
11	2014-01-12	47.0	优	27.0	47.0	27.0	0.7	39.0	59.0	2014	一季度	一级优
19	2014-01-20	35.0	优	8.0	35.0	6.0	0.3	15.0	65.0	2014	一季度	一级优
20	2014-01-21	26.0	优	18.0	25.0	27.0	0.7	34.0	50.0	2014	一季度	一级优
...	...	...	...	...	...	...	...	...	...	...	...	...
2122	2019-10-25	30.0	优	8.0	20.0	2.0	0.4	24.0	55.0	2019	四季度	一级优
2131	2019-11-03	48.0	优	33.0	48.0	3.0	0.6	34.0	33.0	2019	四季度	一级优
2135	2019-11-07	47.0	优	24.0	47.0	3.0	0.5	37.0	44.0	2019	四季度	一级优
2152	2019-11-24	30.0	优	7.0	30.0	3.0	0.2	11.0	58.0	2019	四季度	一级优
2153	2019-11-25	40.0	优	13.0	30.0	3.0	0.4	32.0	29.0	2019	四季度	一级优

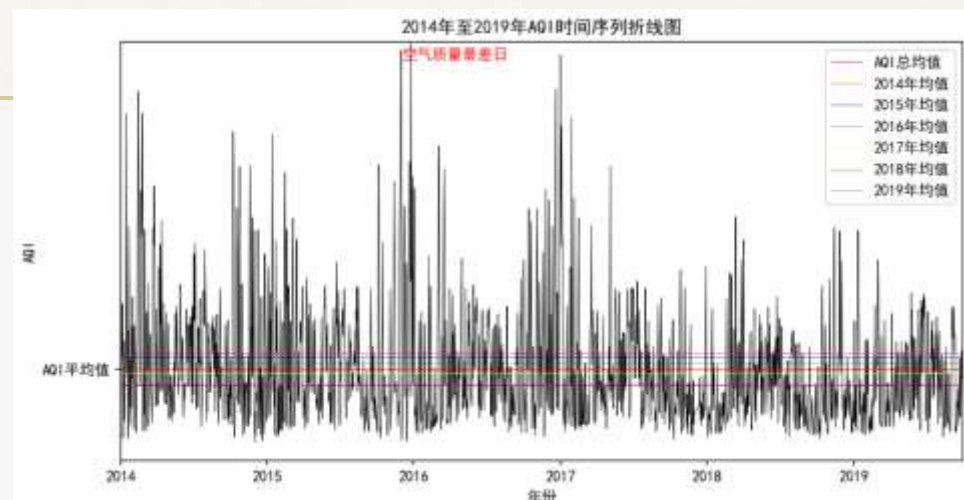
Matplotlib的综合应用：空气质量监测数据的图形化展示

• AQI的时序变化特点

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 %matplotlib inline
5 plt.rcParams['font.sans-serif']=['SimHei'] #解决中文显示乱码问题
6 plt.rcParams['axes.unicode_minus']=False
7 data = pd.read_excel('北京市空气质量数据.xlsx')
8 data = data.replace(0,np.NaN)
9 data = data.dropna()
10 plt.figure(dpi=150,figsize=(10,5))
11 plt.plot(data['AQI'],color='black',linestyle='-',linewidth=0.5)
12 plt.axhline(y=data['AQI'].mean(),color='red',linestyle='-',linewidth=0.5,label='AQI总均值')
13 data['年'] = data['日期'].apply(lambda x:x.year)
14 AQI_mean = data['AQI'].groupby(data['年']).mean().values
15 year = list(data['AQI'].groupby(data['年']).mean().index)
16 col = ['red','blue','green','yellow','purple','brown']
17 for i in range(6):
18     plt.axhline(y=AQI_mean[i],color=col[i],linestyle='--',linewidth=0.5,label=str(year[i])+'年均值')
19 plt.title('2014年至2019年AQI时间序列折线图')
20 plt.xlabel('年份')
21 plt.ylabel('AQI')
22 plt.xlim(xmax=len(data),xmin=1)
23 plt.ylim(ymax=data['AQI'].max(),ymin=1)
24 plt.yticks([data['AQI'].mean()],['AQI平均值'])
25 plt.xticks([1,365,365*2,365*3,365*4,365*5],['2014','2015','2016','2017','2018','2019'])
26 plt.legend(loc='best')
27 plt.text(x=list(data['AQI']).index(data['AQI'].max()),y=data['AQI'].max()-20,s='空气质量最差日',color='red')
28 plt.show()
    
```

Chapter2-4.ipynb

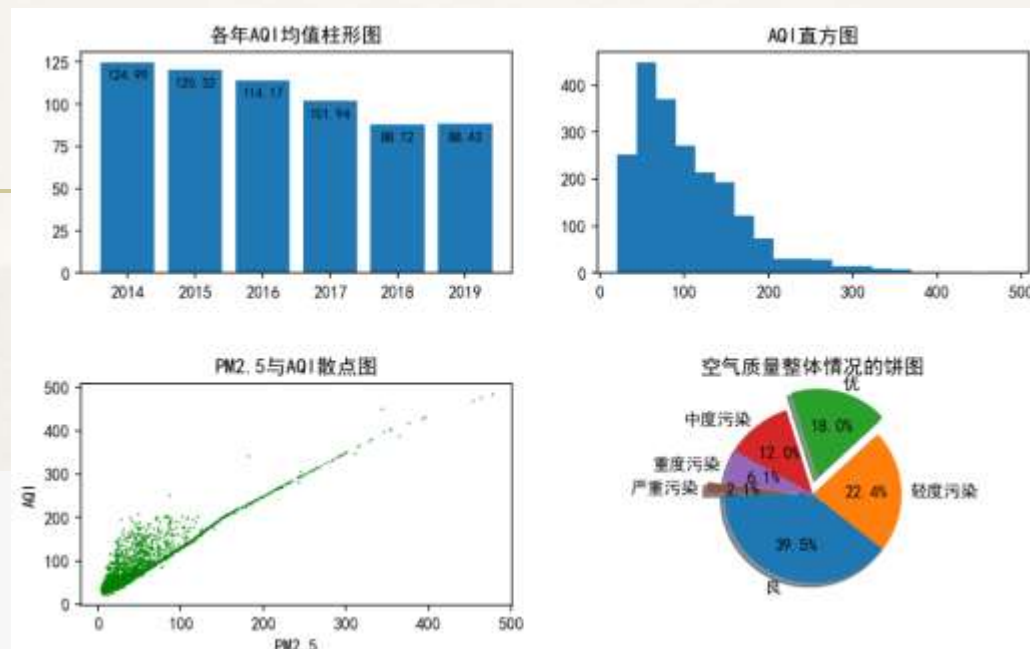


Matplotlib的综合应用：空气质量监测数据的图形化展示

• AQI的分布特征及相关性分析

Chapter2-4.ipynb

```
1 import warnings
2 warnings.filterwarnings(action = 'ignore')
3 fig, axes = plt.subplots(nrows=2,ncols=2,dpi=150,figsize=(10,6))
4 axes[0,0].bar(x=year,height=AQI_mean,data=data)
5 for i in np.arange(len(year)):
6     axes[0,0].text(x=year[i],y=AQI_mean[i]-10, s=round(AQI_mean[i],2), ha='center',fontsize=8)
7 axes[0,0].set_title('各年AQI均值柱形图')
8 axes[0,1].hist(data['AQI'],bins=20)
9 axes[0,1].set_title('AQI直方图')
10 axes[1,0].scatter(data['PM2.5'],data['AQI'],s=0.5,c='green',marker='.')
11 axes[1,0].set_title('PM2.5与AQI散点图')
12 axes[1,0].set_xlabel('PM2.5')
13 axes[1,0].set_ylabel('AQI')
14 tmp = pd.value_counts(data['质量等级'],sort=True) #等同: tmp=data['质量等级'].value_counts()
15 share = tmp/sum(tmp)
16 labels = tmp.index
17 explode = [0, 0, 0.2, 0, 0, 0.2]
18 axes[1,1].pie(share, explode = explode,labels = labels, autopct = '%3.1f%%',startangle = 180, shadow = True)
19 axes[1,1].set_title('空气质量整体情况的饼图')
20 fig.subplots_adjust(hspace=0.5)
21 fig.subplots_adjust(wspace=0.2)
```

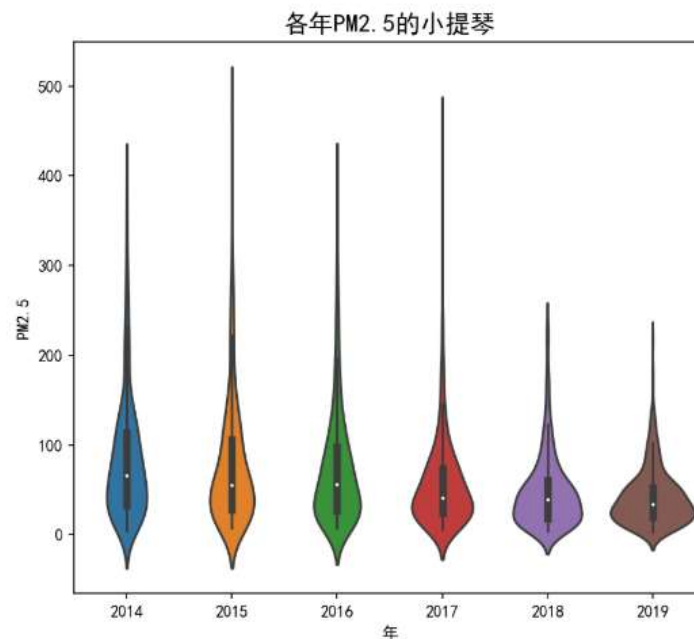
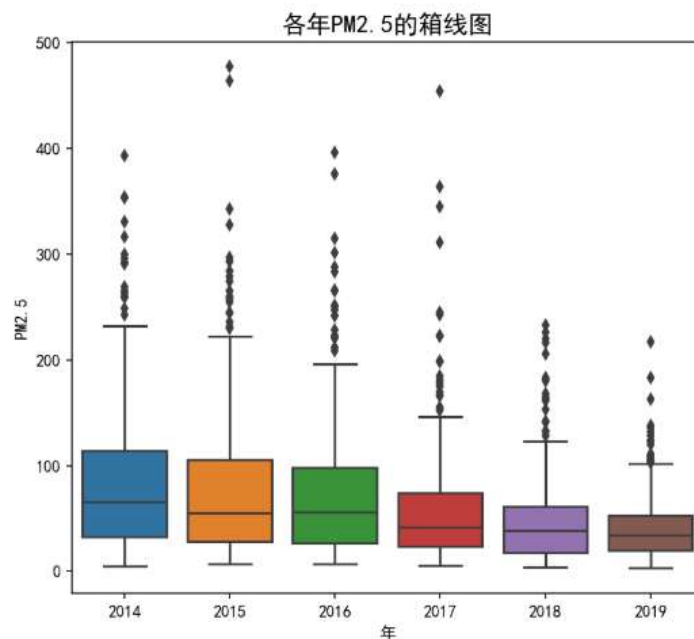
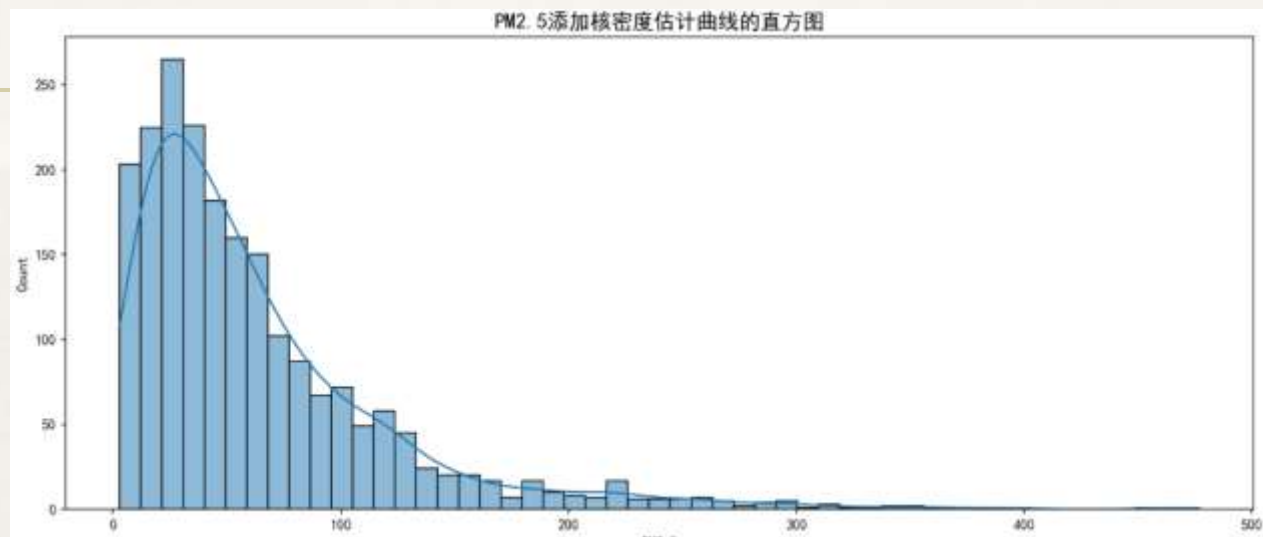


Seaborn的综合应用：空气质量监测数据的图形化展示

• PM2.5的分布特征

Chapter2-4.ipynb

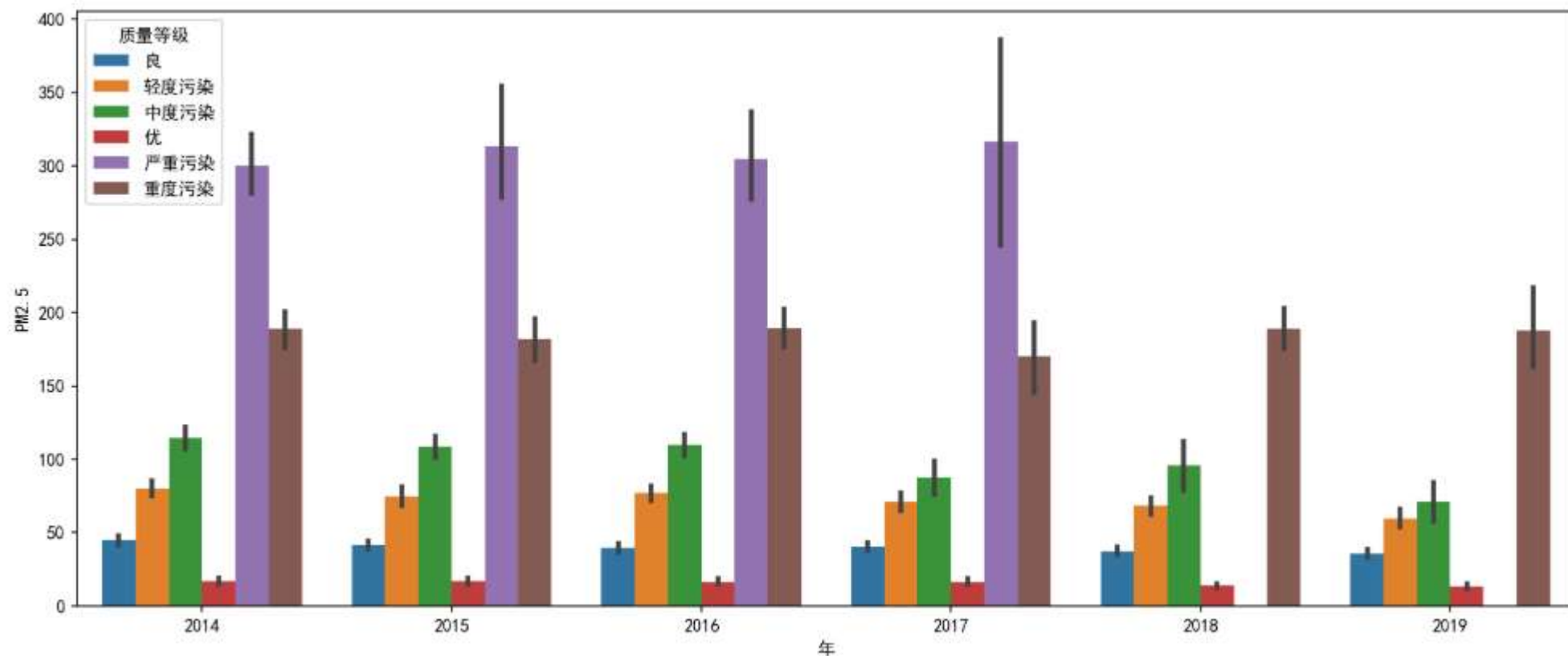
```
1 import seaborn as sns
2 plt.figure(figsize=(15, 6), dpi=150)
3 sns.histplot(data['PM2.5'], kde=True)
4 plt.title('PM2.5添加核密度估计曲线的直方图', fontsize=15)
5 plt.show()
6 plt.figure(figsize=(15, 6), dpi=150)
7 plt.subplot(1, 2, 1)
8 sns.boxplot(x='年', y='PM2.5', data=data)
9 plt.title('各年PM2.5的箱线图', fontsize=15)
10 plt.subplot(1, 2, 2)
11 sns.violinplot(x='年', y='PM2.5', data=data)
12 plt.title('各年PM2.5的小提琴图', fontsize=15)
```



Seaborn的综合应用：空气质量监测数据的图形化展示

- PM2.5的分布特征:各年不同空气质量等级下PM2.5的均值

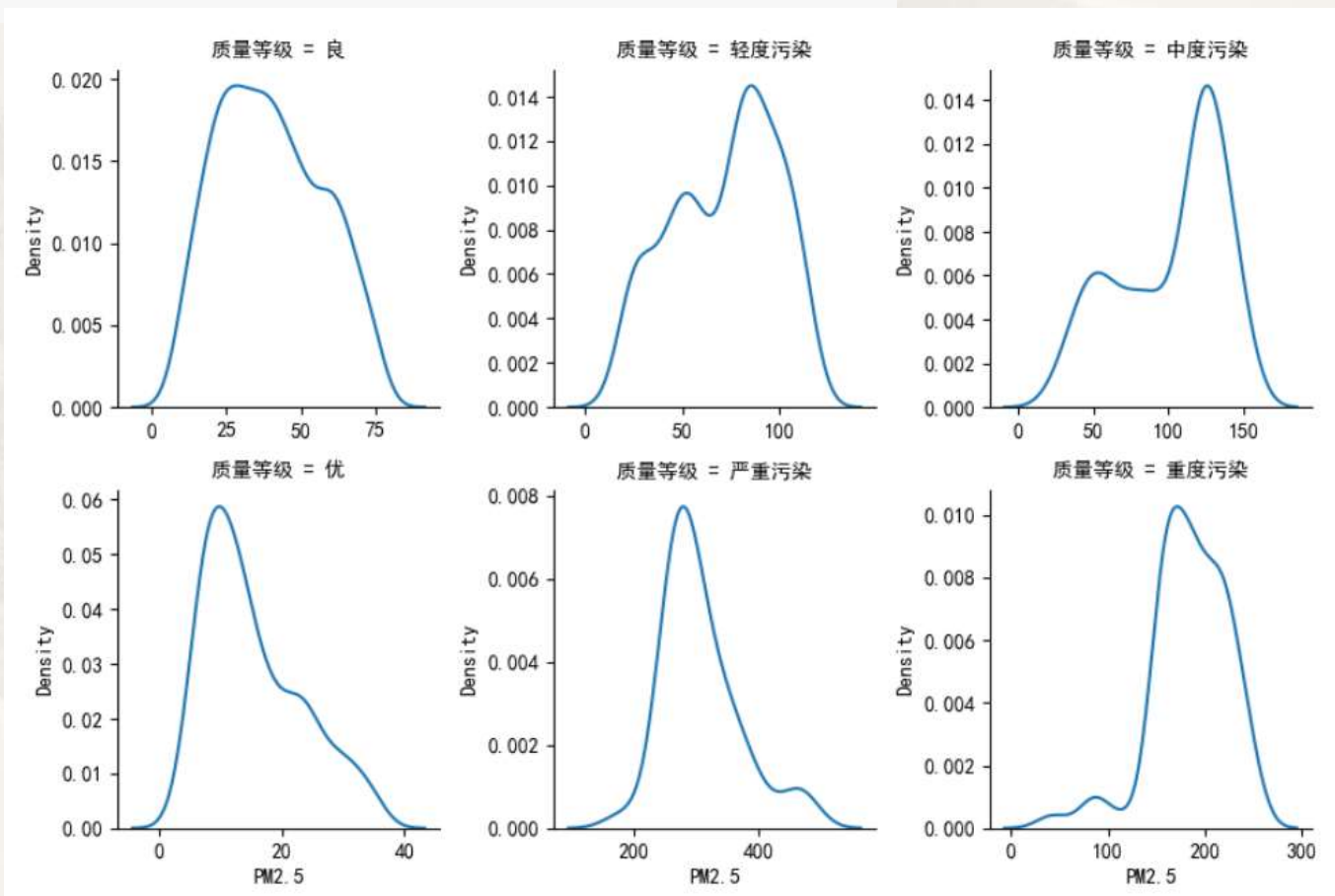
```
plt.figure(figsize=(15,6),dpi=150)  
sns.barplot(x='年',y='PM2.5',hue='质量等级',data=data)
```



Seaborn的综合应用：空气质量监测数据的图形化展示

- PM2.5的分布特征：创建多图网格，并利用核密度曲线图分别展示空气质量不同等级下PM2.5的分布特征

```
1 plt.figure(figsize=(15,6),dpi=150)
2 f = sns.FacetGrid(data=data,col='质量等级',col_wrap=3,sharex=False,sharey=False)
3 f.map(sns.kdeplot,'PM2.5')
```



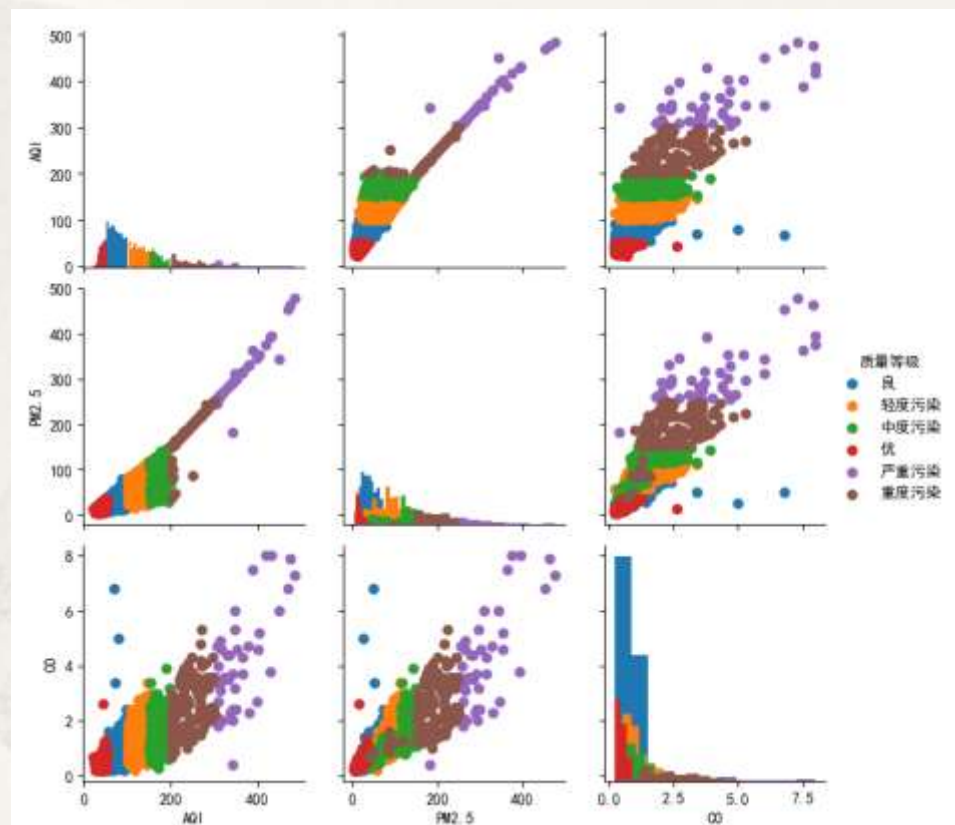
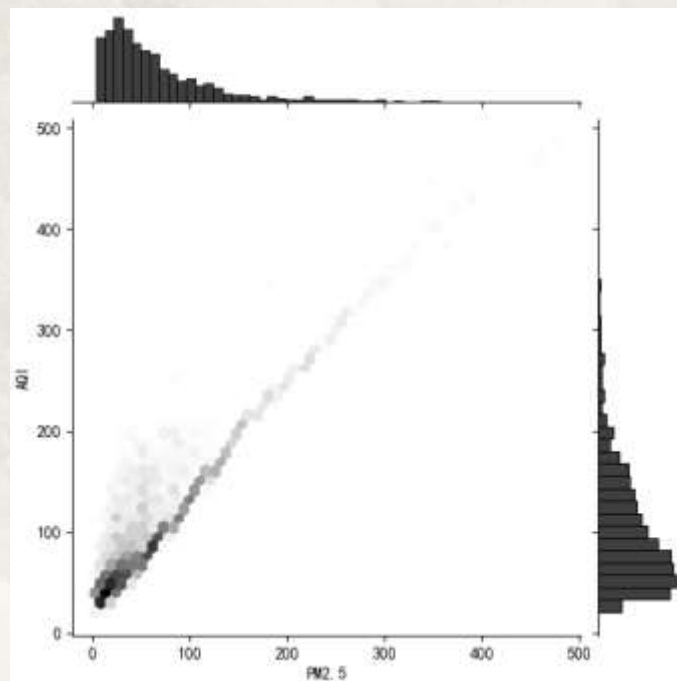
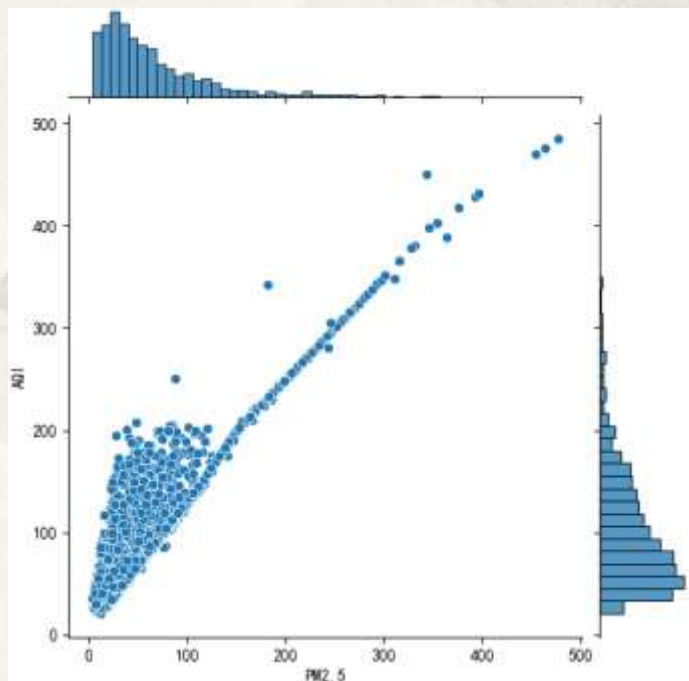
Seaborn的综合应用：空气质量监测数据的图形化展示

• 矩阵散点图：变量相关性

```
1 g = sns.PairGrid(data[['AQI', 'PM2.5', 'CO', '质量等级']], hue='质量等级')
2 g.map_diag(plt.hist) #指定对角线上画什么图
3 g.map_offdiag(plt.scatter) #指定非对角线上画什么图
4 g.add_legend()
```

• 联合图：

```
1 sns.jointplot(x='PM2.5', y='AQI', data=data)
2 sns.jointplot(x='PM2.5', y='AQI', kind='hex', color='k', data=data)
```



作业(自己练习，不交！)

- Python编程(Page41 第3题,Page42 第4题)：学生成绩数据的处理和基本分析
 - 有60名学生的两个课程成绩的数据文件(文件名：ReportCard1.txt, ReportCard2.txt)，分别记录着学生的学号、性别以及不同课程的成绩。请将数据读入Pandas数据框，并做如下预处理和基本分析：
 - 将两个数据文件按学号合并为一个数据文件，得到包含所有课程成绩的数据文件
 - 计算每个同学的总成绩和平均成绩
 - 将数据按总成绩的降序排序
 - 按性别分别计算各门课程的平均成绩
 - 按优、良、中、及格和不及格，对平均成绩进行分组
 - 按性别统计优、良、中、及格和不及格的人数
 - 生成性别的虚拟自变量
 - 学生成绩数据的图形化展示：对包含所有课程成绩的数据文件，做如下图形化展示：
 - 绘制总成绩的直方图
 - 绘制平均成绩的优、良、中、及格和不及格的饼图
 - 绘制总成绩和数学成绩(math)的散点图